

Stack — Practice Questions

Theory + programming questions with fully worked solutions, arranged easy to hard.

CONTENTS

1. Theory: True/False & Short Answer
2. Q1 — Basic Push/Pop Trace
3. Q2 — Push Only Even Numbers
4. Q3 — Reverse a String Using Stack
5. Q4 — Push Words Longer Than N
6. Q5 — Check Balanced Parentheses
7. Q6 — Menu-Driven Stack Program
8. Q7 — Find Maximum Using Stack

01

Theory: True/False & Short Answer

EASY

Q1. A stack follows the FIFO principle. True or False?

Solution

False. A stack follows **LIFO** (Last In, First Out). FIFO (First In, First Out) is followed by a **queue**, not a stack.

Q2. Which Python list method is used to perform the Push operation?

Solution

`.append()` — it adds an element to the end of the list, which we treat as the "top" of the stack.

Q3. What happens when you call `.pop()` on an empty list?

Solution

Python raises an **IndexError** ("pop from empty list"). This is why we always check `if stack:` before popping — to avoid this **Stack Underflow** condition.

Q4. Name two real-life examples of a stack.

Solution

Any two of: a pile of plates, a stack of books, the browser's Back button, the Undo (Ctrl+Z) feature, bangles stacked on a wrist.

02

Q1 — Basic Push/Pop Trace

EASY

Trace the output of the following code:

```
stack = []
stack.append(5)
stack.append(10)
stack.append(15)
stack.pop()
stack.append(20)
print(stack)
```

Solution

Step	Operation	Stack after
1	append(5)	[5]
2	append(10)	[5, 10]
3	append(15)	[5, 10, 15]
4	pop() → removes 15	[5, 10]
5	append(20)	[5, 10, 20]

Output: [5, 10, 20]

03

Q2 — Push Only Even Numbers

EASY

Write a program that reads a list of numbers and pushes only the **even** numbers onto a stack. Then print the final stack.

Solution

```
numbers = [12, 5, 8, 3, 20, 7, 16]
```

```

stack = []

for num in numbers:
    if num % 2 == 0:
        stack.append(num)

print("Stack:", stack)

```

Output: Stack: [12, 8, 20, 16]

Explanation: We loop through every number, use the modulus operator `% 2 == 0` to check if it's even, and only `.append()` (push) numbers that pass the check.

04

Q3 — Reverse a String Using Stack

MEDIUM

Write a program to reverse a string using stack operations (push each character, then pop them all).

Solution

```

word = "PYTHON"
stack = []

# push every character
for ch in word:
    stack.append(ch)

# pop every character to build the reversed string
reversed_word = ""
while stack:
    reversed_word = reversed_word + stack.pop()

print(reversed_word)

```

Output: NOHTYP

Explanation: Since a stack is LIFO, the **last** character pushed (the last letter of the word) is the **first** one popped — which naturally reverses the string. The `while stack:` loop continues until the stack is empty.

Push: P Y T H O N



Fig 1 — Popping in reverse order naturally flips the string

Q4 — Push Words Longer Than N

MEDIUM

Write a program that takes a sentence, splits it into words, and pushes only the words with **more than 4 letters** onto a stack. Then display the stack.

Solution

```
sentence = "Python is a powerful programming language"
words = sentence.split() # splits into a list of words
stack = []

for word in words:
    if len(word) > 4:
        stack.append(word)

print("Stack:", stack)
```

Output: Stack: ['Python', 'powerful', 'programming', 'language']

Explanation: `.split()` breaks the sentence into a list of words (using spaces).
`len(word)` checks each word's length before pushing.

Q5 — Check Balanced Parentheses

HARD

Write a program to check whether the parentheses in an expression are **balanced** — every `(` must have a matching `)` in the correct order. Example: `"(a+b)*(c-d)"` is balanced; `"(a+b))"` is not.

Solution

```
def is_balanced(expr):
    stack = []
    for ch in expr:
        if ch == "(":
            stack.append(ch)
        elif ch == ")":
            if not stack:
                # nothing to match with
                return False
            stack.pop()
            # matched, remove it
    return len(stack) == 0 # True only if fully matched

print(is_balanced("(a+b)*(c-d)")) # True
print(is_balanced("(a+b))"))      # False
print(is_balanced("((a+b))"))     # False
```

Explanation:

- Every (is **pushed** onto the stack.
- Every) tries to **pop** a matching (. If the stack is empty at that point, there's an extra) with nothing to match — unbalanced.
- At the end, if the stack is **not empty**, some (was never closed — also unbalanced.

Why stack fits perfectly: The most recently opened bracket must be the first one closed — that's exactly the LIFO behavior a stack gives us.

07

Q6 — Menu-Driven Stack Program

MEDIUM

Write a menu-driven program that allows the user to Push, Pop, and Display a stack of integers, repeating until they choose to exit.

Solution

```
stack = []

while True:
    print("\n1. Push  2. Pop  3. Display  4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value to push: "))
        stack.append(value)
        print(value, "pushed successfully")

    elif choice == 2:
        if not stack:
            print("Stack Underflow! Stack is empty")
        else:
            print(stack.pop(), "popped successfully")

    elif choice == 3:
        if not stack:
            print("Stack is empty")
        else:
            print("Stack:", stack)

    elif choice == 4:
        print("Exiting program")
        break

    else:
        print("Invalid choice, try again")
```

Explanation: The `while True` loop keeps showing the menu until the user picks

option 4, which uses `break` to exit. Each choice calls the matching stack operation, with an underflow check before popping.

08

Q7 — Find Maximum Using Stack

HARD

Write a program that pushes a list of numbers onto a stack, then pops all elements one by one to find and print the **maximum** value — without using the built-in `max()` function.

Solution

```
numbers = [23, 67, 12, 89, 45, 9]
stack = []

for num in numbers:
    stack.append(num)

largest = stack.pop() # start with the last element

while stack:
    current = stack.pop()
    if current > largest:
        largest = current

print("Maximum value:", largest)
```

Output: Maximum value: 89

Explanation: We first push every number, then pop them one at a time, keeping track of the biggest value seen so far by comparing it to `largest` at each step.

How to use this set: Try solving each question yourself first (cover the solution box), then check your code against the given answer. Questions are ordered from basic tracing to full logic-building — the same range CBSE board papers typically test.